ZeroRun: Reproducible test-result reuse for AI coding tools

Jishan Kapoor*

Independent researcher, Toronto, Canada

Abstract

ZeroRun provides auditable validation handoffs for AI coding tools. It distinguishes an earlier successful test status under a declared source/runtime identity from fresh execution for diagnostics. The software combines repository-bound CLI/MCP interfaces, authenticated records, a hit path without source staging, and an executable evaluation kit. A corrected-fixture study completed 24 selected cases across eight repositories, with 48 reused successes agreeing with fresh checks. Scripted request latency decreased; total cost varied and host timing remains qualified. Fresh checks verified five of six model-produced fixes; 15 guided model-consumer turns distinguished prior status, fresh success and fresh failure. These observations support qualified status handoffs, not general workflow acceleration. Input completeness and determinism remain operator assumptions; reused status neither replays output nor proves patch correctness.

Keywords: AI coding tools, test-result reuse, reproducibility, software testing

*Corresponding author.
Email address: kapoorjishan2@gmail.com (Jishan Kapoor)

Required Metadata

Current code version

Nr.	Code metadata description	Value
C1	Current code version	ZeroRun 0.5.3; research release 2026-09-08; historical study runtime 0.5.1 retained separately
C2	Permanent GitHub link to code/repository used for this code version	`floxy-21/zerorun-research`; linked immutable source/evidence snapshot
C3	Legal Code License	MIT (ZeroRun code); third-party licenses retained; trajectory data CC-BY-4.0
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python; pytest; Docker; CLI; Model Context Protocol (MCP); Codex skill
C6	Compilation requirements, operating environments & dependencies	Runtime: Python $\geq$3.10; no core Python dependencies. Whole-task reuse: Linux/amd64 and Docker. Canonical analysis: CPython 3.12–3.14. Experiments: digest-pinned Python 3.10/3.12 containers and frozen dependencies.
C7	If available Link to developer documentation/manual	Version-pinned README: installation, interfaces, reproduction, and limitations
C8	Support email for questions	kapoorjishan2@gmail.com

Table 1: Code metadata. The public release preserves the tested runtime source and provides a separate, documented research harness.

1. Motivation and significance

AI coding tools use test results to assess edits, as illustrated by SWE-bench and SWE-agent [1, 2]. After a producer validates a patch, another tool or reviewer needs to distinguish identified earlier success from newly executed evidence: identical commands can observe different source, dependencies or external state.

ZeroRun exposes a deterministic command's previous successful status under an unchanged declared identity. Testing-tool integrators and researchers can request that status or fresh diagnostics, with separate provenance and costs. Regression-test selection avoids tests considered unaffected by a change [3, 4]. Existing build and LLM-tool caches already address result reuse (Table 2); eidos provides adjacent LLM function-integration software [5]. LabChain combines hash-based intermediate-result reuse with modular scientific pipelines and JSON configuration [6]. ZeroRun instead exposes an earlier test-status decision under an operator-qualified execution identity. Neither content hashing nor attaching a cache to an AI interface is a new algorithm here.

The contribution is an inspectable validation handoff for operator-qualified, result-only tests: a repository-bound interface distinguishing historical from fresh evidence, external authorization, a hit path avoiding source staging, and an evaluation kit connecting input boundaries, independent fresh outcomes and full request cost. Researchers can therefore investigate qualification and downstream waiting without conflating a cached success with a fresh test.

A saved receipt can document a past run. ZeroRun additionally packages declared-input revalidation, authenticated lookup, authorization, and explicit fresh-verification behavior through one interface. A receipt system implementing the same checks could provide comparable functionality; no performance advantage over such an implementation is claimed.

2. Software description

2.1. Architecture and execution contract

Figure 1 shows the execution path. A version-2 whole-task manifest declares the command, input paths, environment policy and digest-pinned Linux/amd64 container. The contract requires a result-only computation without output artifacts or cached streams. Input completeness and determinism require operator review; passing tests cannot establish them.

The identity includes declared content, relevant command-source content, directory modes, command, environment policy and inspected runtime. Entries must authenticate and satisfy the contract. Misses execute private source copies read-only in resource-constrained, network-disabled containers; source

System	Reuse basis	Returned information / execution state
Bazel [7]	Build-action hashes with declared inputs and outputs	Action metadata, output files, stdout and stderr.
ToolCaching [8]	Tool name and serialized arguments; feature/TTL-based admission	Tool results and metadata; requests classified as COMMAND excluded.
TVCache [9]	Tool-call prefixes; optional state-preserving annotations	Tool values; selective sandbox snapshots support continued rollout execution.
ZeroRun	Reviewed local test inputs, command and runtime identity	Identified earlier success; no historical transcript or restored output artifacts.

Table 2: Operating contracts, not a performance ranking. ZeroRun evaluates a narrower result-only task contract; no head-to-head superiority is established.

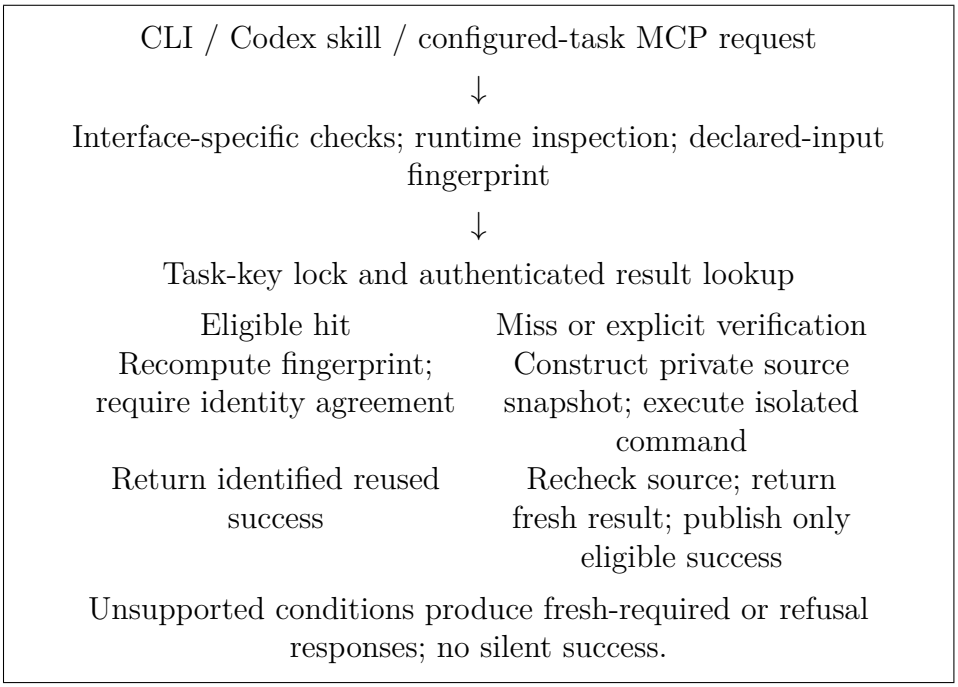

Figure 1: Whole-file result-reuse path. A hit returns status metadata, not the prior transcript. This explanatory diagram was revised as editable LaTeX with OpenAI Codex desktop 26.901.6511.0 assistance and checked against the described implementation; it is not experimental image data.

drift is checked before publishing success. Failures are not reused. Explicit verification can quarantine a stored result conflicting with fresh execution.
The optimized whole-file path fingerprints inputs, locks and authenticates the entry, then independently fingerprints again. A hit requires both identities and payloads to agree; it neither executes repository code nor stages a copy. Misses, forced execution, verification and symbol-projection tasks retain snapshot execution. Eligibility is unchanged.
This is conditional correctness, not a soundness proof for arbitrary Python. An undeclared dependency, nondeterministic test, or violated host assumption can invalidate reuse. Two filesystem scans are not an atomic snapshot against arbitrary concurrent host mutations. Unsupported or insufficiently reviewed work must execute fresh or remain refused.

2.2. Interfaces and functionality

The CLI exposes structured results identifying reused success, fresh success, fresh failure, uncertainty, and refusal. The Codex skill and repository-bound MCP server expose the same distinction. Normal agent-facing reuse additionally requires exact-configuration authorization outside repository-controlled files; generating observation candidates does not authorize reuse. Laboratory fixtures used by the experiments authenticate cache records but are not production operator reviews, and their private keys are excluded from the artifact.
The current skill routes reviewed whole-task configurations first; optional fine-grained pytest reuse requires fresh execution when qualification is incomplete, because omitted tests can affect fixtures, callbacks, shared state or ordering. It is not this article's demonstrated acceleration mechanism. Whole-task reuse requires the complete command to satisfy the contract, not independence of individual tests.
Hits do not reconstruct stdout or stderr. Clients requiring new diagnostics, coverage, or generated files must execute fresh. The public operating guide and manifest reference document installation, operator review, authorization, and this boundary; automated interface checks do not establish external adoption. On a hit, `execution_ms` describes the earlier execution; `wall_ms` describes this runner request. Clipped `saved_ms` is an estimate, not measured net workflow savings.

3. Illustrative examples

3.1. Repeat, failure, and restoration

The executable example starts with an empty store and a fixed pytest target. It executes a seed, reuses unchanged successes, executes an added failure

twice, then finds the earlier success after byte restoration. The seven-response sequence is:

Request	Expected whole-task response
Seed	`MISS_EXECUTED`
Two unchanged repeats	`HIT_REUSED` each
Added failure; repeated failure	`MISS_FAILED` each
Restoration; restored repeat	`HIT_REUSED` each

The example records an independent fresh full-target check after every request, including hits. It is a constructed validation sequence, not an estimate of the repetition frequency in real development.
For a configured target named `tests`, the CLI form is:

```
zerorun --manifest .zerorun.json --json explain tests
zerorun --manifest .zerorun.json --json run tests
zerorun --manifest .zerorun.json --json run tests --verify
```

These commands inspect eligibility, request execution/reuse, and verify fresh. The operator guide supplies review and authorization; commands alone cannot establish input completeness.

3.2. Client decisions and integration evidence

The reference consumer checks response consistency, `status`, `exit_code` and protocol errors; `mode=reuse` alone is not success. It labels historical success and routes fresh failures or requests to execution, without authorizing tasks or executing its recommended fallback.
For the same configured task, MCP `run_tests` takes the following arguments for ordinary reuse eligibility and fresh verification, respectively:

```
{"task":"tests"}
{"task":"tests","verify":true}
```

The server is bound to the target repository and receives the operator's external authority location. Requesting reuse does not imply that the response is a hit.
Abbreviated fields from the synthetic 0.5.3 quickstart receipt make the choice inspectable: `{"status":"HIT_REUSED","exit_code":0}` identifies earlier success; `{"status":"VERIFY_MATCH","exit_code":0}` reports fresh agreement. A consumer accepting prior status can continue; one needing fresh diagnostics requests verification and inspects its fresh output.
The historical client-decision trials use a deterministic two-file synthetic fixture, separately from the real issues below. Three Codex configurations

failed overall through missing authority, approval refusal or unsupported arguments. A separate API-guided demonstration passed two model decisions and six interpretations without retries; neither an identified backing model nor a causal documentation effect was established. Full trial denominators, failures, scripted checks and fresh oracles remain in the supplement. Separately, a fresh anonymous 0.5.3 public clone passed external installation and five actual MCP stages: refusal, readiness, fresh execution, reuse and fresh verification, with 36 matching runtime files. Installation took 16.3 s and server checks 11.6 s, excluding cloning, Docker setup and preparation. This account-free synthetic check is not an independent user study.

3.3. Actual model-produced repairs and consumer handoffs

A prospective application selected six issues across six repositories. Each producer received the issue and unchanged supplied tests, without the reference production fix, for one bounded model attempt. Fresh container checks of each baseline and actual final snapshot verified five fixes with identical collected-node sets. SQLGlot's primary patch retained two failing dialect tests; a separately assisted production repair subsequently passed the same 99-node target without changing the original 5/6 outcome. Requests specified gpt-6-astra, medium effort, through Codex CLI 0.153.3; captured events did not identify the backing model independently.

Laboratory scripts qualified, externally authorized and seeded the five eligible patched states. Actual model consumers then received task-level requests about available evidence, newly executed diagnostics, and an operator-restored original buggy state. All 15 turns completed, each with one MCP request: five `HIT_REUSED`, five `VERIFY_MATCH`, and five fresh `MISS_FAILED` results after restoration. Every model correctly distinguished historical status from new execution and success from failure within the configured target. This is a guided application using model-produced patches, with scripted preparation and restoration; it is not a wholly autonomous workflow.

The frozen parser left five failed client envelopes unclassified; a separate additive audit verifies their intact fresh-failure payloads without rewriting original records. Internal AI-assisted review confirmed all 15 core status/freshness interpretations. Thirteen messages additionally asserted wire error flags omitted from the CLI archive; those details remain unverifiable, not certified or established false. Complete prompts, transcripts, preparation failures and separate cost intervals are in the application ledger. The 415.91 s consumer-model total includes 60.68 s runner time; neither is a controlled end-to-end saving.

Worked qualification. The Pycparser record binds `tests/test_c_parser.py`, parser modules, generated tables, fixtures and configuration under the fixed image, with no forwarded host environment or reusable outputs. Inspection covers exactly the actual patched state and restoration of `pycparser/c_parser.py`; tests remain unchanged. Any other source, command, dependency or contract change requires renewed review. `closure_reviewed` records an assertion, not proven completeness or determinism. Expert review effort remains unmeasured.

3.4. Issue-state validation handoffs

A frozen issue-state cohort contains 24 cases across eight repositories, selected by repository shortlist, metadata checks and hash ordering before timing. Both arms receive the same reference-patched source and supplied full target. Fresh-only executes a producer and consumer; ZeroRun executes a cold producer then requests identified prior status. Each arm is checked against an independently fresh full-target oracle. Two blocks reverse arm order. These imposed handoffs do not measure natural repetition frequency.

The corrected-fixture V6 repeat completed all 24 cases across 8 repositories: 48 paired blocks, 48 reused successes and 96 agreeing fresh-oracle checks. Complete chain time was 4.1% lower, and mean scripted consumer-request latency 78.9% lower. Aggregate chains improved in 15/24 cases and worsened in 9/24. This demonstrates bounded operation and workload-dependent costs, not population-wide acceleration. Source inventories and the actual runtime image are bound before and after execution.

	Cases	Chain sum (s)		Consumer mean (s)	
Deployment	complete/selected	Fresh	ZeroRun	Fresh	ZeroRun
Copy pilot	2/2	42.79	60.89	5.17	2.08
Image repeat	2/2	73.76	81.67	8.82	2.53
Compatible V5	23/24	350.13	320.31	3.67	0.69
Corrected V6	24/24	564.56	541.17	5.63	1.19

Table 3: Separate cohorts, without pooling. Chain includes cold producer plus consumer, excluding per-arm setup, common image construction, operator review, fresh oracles and diagnostics. Measured setup, oracle and diagnostic costs are retained separately; operator review is unmeasured. Earlier interrupted campaigns remain in the supplement. V6 timings are descriptive, with the host-monitor qualification below. Its corrected Lizard fixture and original V5 failure are retained.

Adding per-arm setup gives V6 totals of 572.54 s fresh and 545.46 s ZeroRun. Common compatible-image preparation took 82.29 s separately. Its recorded

Docker build disabled layer-cache reuse and network access using 16 hash-bound wheels; the existing VM and base image remained prerequisites. This establishes an author-side recipe build, not clean-OS or independent-human replication, and the loopback-derived image is not a publicly pullable artifact. V5 completed 23 cases and retained Lizard-191 as unsupported. V6 preserves every selected case, command and source patch, changing only its stale expected complexity from two to three; the exact public upstream method supports that correction. This intervention is explicit, not a replacement of an unfavorable case. The original 2/24 partial main campaign and interrupted 15/24 repeat remain separate in the coverage audit, with compatibility, collection, lifecycle, assertion and capture failures distinguished from reuse errors. No incomplete case contributes a zero-cost observation. The original fine-grained V6 host-monitor files were not recovered. The retained VirtualBox log contains a 5.53 s heartbeat lapse within the nominal run window, without an explicit pause transition. Its clock association is conditional; V6 does not certify uninterrupted or quiet-host performance.

The image repeat reduced consumer waiting 71.3% while increasing chain time 10.7%. A consumer needing an identified earlier status can benefit after producer work has finished; a consumer requiring newly executed diagnostics must pay for fresh validation. Qualification and acquisition are additional costs, not assumed free. The operating guide specifies declared inventory, exclusions and re-review triggers; an unchanged manifest does not qualify arbitrary future implementation changes.

The coverage audit and reproduction guide index every cohort, raw outcome, setup cost and intervention. The earlier single verified model-producer pilot remains separate from these reference-patch comparisons and the new installed-client application.

An exploratory follow-up selected LKML-85 after observing V6's cold overhead. With one consumer, both chains were slower (56.6–365.6%); two consumers gave mixed results; four gave 31.0–32.6% lower chain cost. All six blocks and fresh checks are retained. Fixed condition order and concurrent host I/O prevent separating repetition from warm-up and host effects. This sensitivity example is not pooled with V6 or evidence of natural demand. Net time benefit must subtract additional setup, qualification and execution costs from avoided repeat execution.

3.5. Real AI-workflow observations

The read-only audit uses public SWE-rebench OpenHands records [10, 11]. After ten pilot rows, fixed SHA-256 ordering selected 128 of 67,064 remaining rows without outcome or performance filtering. Raw responses, exclusions, the parser and same-selection rate-limit recovery are retained.

Of these, 122 paired episodes span 104 repositories and 120 issues; six unmatched episodes were excluded without replacement. There were 745 supported direct pytest invocations and 120 exact-command repeats, all separated by barriers: 112 editor mutations and eight other unmeasured effects. These are not cache hits. Compound commands are excluded; clipped outputs and missing exit markers remain visible.
Repeated command text cannot establish source/runtime identity, even after apparent restoration. A purposively selected django-environ episode provides a positive, bounded mechanism example. We reconstructed its recorded source edits, added a fresh check at the intermediate edited state, and ran four controlled requests (Table 4). After the inverse edit, the complete declared source/runtime identity and original target's cache key rejoined; the restored request returned one explicitly identified reused success, agreeing with an independent fresh 14-test outcome. The intervening collection command selected no tests and exited 5; it remained a fresh failure, not a reusable success.

Controlled request	Actual response	Exit	Fresh nodes
Seed target	`MISS_EXECUTED`	0	14
Edited target (extra check)	`MISS_EXECUTED`	0	15
Restored collection	`MISS_FAILED`	5	0
Restored target	`HIT_REUSED`	0	14

Table 4: One source-restoration case. The 15-test request is additional counterfactual instrumentation, not a test call made by the recorded agent. Every row includes a separate fresh full-target oracle.

This standalone case ran after the timing VM interruption, with independently checked source bindings. Retained failed attempts exposed a missing Git-fixture marker; a separately versioned producer initialized local metadata while preserving the baseline's masking guard. The successful run uses Python 3.12.14 and pytest 9.1.1, not the recorded Python 3.9 environment. It demonstrates this content-restoration mechanism, not historical output replay, unchanged autonomous-agent behavior, or population-wide benefit.

4. Impact

4.1. Reproducible validation and measured cost

Researchers can vary targets and input boundaries while retaining fresh-oracle comparisons, source identities, failures, setup costs and explicit denominators.

Codex assisted software, experiment and analysis implementation and manuscript preparation. Separately implemented validators recompute outcomes from retained raw records; input-inventory and fresh-execution oracles do not substitute cached outcomes for their reference. These are automated cross-checks, not independent human replication.

The original comparison used five pinned Python libraries: packaging, pycparser, Colorama, Click, and more-itertools. Ordinary pytest is the uncached reference; disabling only the optimized hit path gives a snapshot-first ablation, not an independent competing cache. Two reversed-order seven-request sequences per library yielded 70 fresh-result agreements and 40 optimized-arm whole-task hits. The original mean conditional hit latency reduction against the ablation was 67.3–71.0%. Complete-sequence direct/optimized ratios were 0.85–2.09. For more-itertools, optimized execution was 16.6% slower overall than snapshot-first reuse, despite faster hits. These results are retained, including large lifecycle delays and their order sensitivity.

The replication used six independently initialized blocks on each previously observed short subject, covering every ordering of direct pytest, snapshot-first and optimized reuse. Seven request states, the 900-second bound and container limits were unchanged. Timings include complete helper/API and Docker lifecycle costs; telemetry and receipt writing are excluded, and preparation retained separately. These are seen workloads, not a holdout or replacement for more-itertools.

All 168 requests in 24 completed planned blocks agreed with fresh full-target checks: 96 optimized hits and 72 non-hits. All 504 arm invocations and 168 fresh-oracle captures are retained. A VM crash after 21 complete blocks required an explicit recovery of only unfinished preselected Packaging blocks four through six, using the unchanged driver. The supplement retains failures and recovery records; coverage is recovered, not uninterrupted. Packaging's unflushed setup total is unavailable and its setup-inclusive ratio omitted. The interrupted attempt retains 1 complete request, 1 partial request and an outcome-less invocation. Including its surviving arm costs changes Packaging's direct/optimized ratio to 1.029.

A post-hoc operating-region analysis retains every completed block and holds within-category composition fixed. Let D_h, F_h and D_m, F_m denote mean complete direct/optimized costs in hit and non-hit categories. For a hypothetical hit fraction p, fixed category means give the equality point

$$p^* = \frac{F_m - D_m}{(F_m - D_m) + (D_h - F_h)}.$$

The crossovers for Click, pycparser, Colorama and Packaging are 54.25%, 63.58%, 56.30%, 55.06%, respectively; larger fractions favor reuse in this

Subject	D/F	D/F + setup	Block range	Hit vs. snapshot (%)
click	1.05	1.05	0.89–1.64	68.1
pycparser	0.89	0.91	0.78–1.04	68.8
colorama	1.01	1.01	0.98–1.11	67.6
packaging	1.04	—	0.88–1.32	70.8

Table 5: Complete replicated sequence cost versus conditional hit benefit. D/F is summed direct time divided by summed optimized time; above one favors reuse. Setup-inclusive ratios allocate common dependency preparation and each block's materialization to both arms. Block range retains all six complete sequences. Hit saving compares median optimized and snapshot-first hit latency, not end-to-end agent time.

fixed-mixture calculation. The imposed fraction is 57.14%, explaining why full-sequence ratios can lie near or below one despite fast hits. All six block-level crossovers per subject are archived; their ranges are descriptive, not confidence intervals. The two fingerprints account for 73.0–75.9% of optimized hit time; their inclusive enclosing timer is not double-counted. This is not a measured agent hit frequency or a deployable admission policy. Different miss/failure mixtures and host costs change the threshold. Common setup cancels from equality; additional deployment/review costs remain unmeasured, not zero.

An independent standard-library input-inventory oracle checks boundary changes without importing ZeroRun's fingerprint implementation. Linux validation covers 32 production/oracle comparisons and six explicit oracle-scope refusals. Cases include same-size content changes with restored modification time, helper/configuration/data edits, additions, deletion, renaming, directory modes, restoration, command and environment changes, missing inputs, and symlinks. A new *undeclared* top-level file changes neither declared identity: the test documents this limitation rather than claiming automatic closure discovery. These checks establish implementation behavior within the tested boundary, not universal determinism.

The supplement retains historical Python 3.10–3.14 CI, Windows and public-package checks with source bindings and skips. A 100-repository interface exercise returned 99 passes and one safe refusal, not 100 upstream-suite or autonomous-task executions. A separate 200-comparison evaluation accepted no fine-grained node reuse; its performance gates remain unmet. These counts are not pooled into a reliability percentage.

4.2. Scope, reuse, and limitations

Integrators can inspect the result contract and review boundary before enabling reuse; researchers can audit responses against implementation bytes,

fresh outcomes, and full cost. The MIT release permits adaptation and commercial reuse of ZeroRun-owned code; third-party licenses remain separate.
No task-reuse prevalence, population speedup, token savings, improved patch quality or productivity is established. Zero barrier-free trajectory pairs neither establish eligibility nor exclude restoration. Library targets are convenience workloads on one constrained VM; original and replicated observations are not independent datasets. The shared Windows host was not fully isolated. Block spreads and influential observations are retained; nested test nodes are not independent samples.
No external users, production deployments, commercial demand or third-party publications using ZeroRun are claimed. Model sessions and controlled handoffs are separate; generalization across clients/models and end-to-end workflow benefit remain unmeasured.

5. Conclusions

ZeroRun contributes an inspectable validation handoff for operator-qualified, result-only tests: explicit historical-versus-fresh status, external authorization, a staging-free hit path and an executable evaluation kit. Consumer waiting can fall while chain time rises; selected application cases establish bounded feasibility rather than population acceleration. Qualification effort and broader autonomous-workflow benefit remain open research questions.

Data and software availability

Code, research data and reproduction instructions are deposited in the public ZeroRun repository [12]. Table 1 pins the submitted distribution. Its manifest separately retains the historical 0.5.1 study runtime at revision `86f42289c2f59a74b1f642f0b20d1a26b3d55e57`; the current 0.5.3 distribution adds whole-task-first integration, narrow custom-authority configuration and timing documentation. Its computational engine is unchanged, with separate source, installation and interface checks. Historical timings are not relabeled as a 0.5.3 replication. Supplementary material 1, `ZeroRun_SoftwareX_reviewer.zip`, contains inventoried software, raw evidence and analysis tools. Public trajectory responses retain CC-BY-4.0 notices and source attribution. Private cache-authentication keys and operator authority are excluded.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Jishan Kapoor: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing–original draft, Writing–review & editing, Resources, Project administration.

Declaration of competing interests

Jishan Kapoor develops ZeroRun and may explore future commercialization if the research progresses. No external funding or established commercial adoption is reported.

Declaration of generative AI and AI-assisted technologies

OpenAI Codex assisted software and experiment implementation, analysis, literature checking, manuscript drafting, and document preparation. The sole human author reviewed and approved the article and its disclosures and takes responsibility for the evidence, references and claims.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, SWE-bench: Can language models resolve real-world GitHub issues?, in: The Twelfth International Conference on Learning Representations, 2024.
URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, SWE-agent: Agent-computer interfaces enable automated software engineering, in: Advances in Neural Information Processing Systems 37, 2024. doi:10.52202/079017-1601.

[3] G. Rothermel, M. J. Harrold, A safe, efficient regression test selection technique, ACM Transactions on Software Engineering and Methodology 6 (2) (1997) 173–210. doi:10.1145/248233.248262.

[4] S. Yoo, M. Harman, Regression testing minimization, selection and prioritization: A survey, Software Testing, Verification and Reliability 22 (2) (2012) 67–120. doi:10.1002/stvr.430.

[5] J. F. Aldana-Martín, A. Benítez-Hidalgo, J. F. Aldana-Montes, eidos: A modular approach to external function integration in LLMs, SoftwareX 31 (2025) 102290. doi:10.1016/j.softx.2025.102290.
URL https://doi.org/10.1016/j.softx.2025.102290

[6] M. Couto, J. Parapar, D. E. Losada, LabChain: Enabling reproducible and modular scientific experiments in Python, SoftwareX 33 (2026) 102543. doi:10.1016/j.softx.2026.102543.
URL https://doi.org/10.1016/j.softx.2026.102543

[7] Bazel Project, Remote caching, official documentation; accessed 2026-09-05 (2026).
URL https://bazel.build/remote/caching

[8] Y. Zhai, D. Shen, J. Luo, B. Yang, ToolCaching: Towards efficient caching for LLM tool-calling, preprint, version 1 (2026). arXiv:2601.15335, doi:10.48550/arXiv.2601.15335.
URL https://arxiv.org/abs/2601.15335

[9] A. V. Kumar, B. Kataria, B. Oh, E. Manzoor, R. Singh, TVCache: A stateful tool-value cache for post-training LLM agents, preprint, version 1 (2026). arXiv:2602.10986, doi:10.48550/arXiv.2602.10986.
URL https://arxiv.org/abs/2602.10986

[10] M. Trofimova, A. Shevtsov, I. Badertdinov, K. Pyaev, S. Karasik, A. Golubev, OpenHands trajectories with Qwen3-Coder-480B-A35B-Instruct, nebius dataset and accompanying blog, CC-BY-4.0; authors and year from the archived dataset citation. Observed revision 35455389ab51bf5e2306bfd436ef72d0f98bf882; accessed 6 September 2026. Exact retrieved responses archived with the article (2025).
URL https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories

[11] I. Badertdinov, A. Golubev, M. Nekrashevich, A. Shevtsov, S. Karasik, A. Andriushchenko, M. Trofimova, D. Litvintseva, B. Yangel, SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, in: Advances in Neural Information Processing Systems 38, 2025. doi:10.52202/085713-0788.
URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/21bec6ace947b1b58967b945c8ac0f10-Abstract-Datasets_and_Benchmarks_Track.html

[12] J. Kapoor, ZeroRun: Research software and source-bound evaluation data, [dataset and software] GitHub, distribution version 0.5.3 with historical 0.5.1 study runtime retained; immutable source/evidence revision identified in the article's code metadata. Accessed 8 September 2026 (2026).
URL https://github.com/floxy-21/zerorun-research

Current executable software version

ZeroRun 0.5.3, Python command-line package. The public release includes installation instructions, automated checks, and the separately identified historical 0.5.1 study runtime. Linux/amd64 Docker execution is required for the demonstrated whole-task reuse mode.